Дисциплина «Технологии разработки программного обеспечения»

# ОТЧЁТ
# ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

## Создание и ведение Git-репозитория

Сквозной проект «Система учёта заявок»

Выполнил: Гатин Тимофей
Группа: еозИСИП-308
Форма обучения: заочная

03.11.2026

# 1. Цель и настройка репозитория

**Цель работы:** организовать разработку проекта с использованием Git.

Проверена версия git version 2.47.1.windows.1. Создан локальный репозиторий Гатин_Тимофей_еозИСИП-308_RequestSystem с основной веткой main. Автор настроен только внутри проекта:

```
git init -b main
git config user.name "timbr0"
git config user.email "timbr0@mail.ru"
```

Имя и e-mail являются подписью авторства коммитов, а не учётными данными для входа на GitHub. Созданы README.md и .gitignore. Игнорируются .venv, IDE, кэш Python, .env, ключи и локальные базы. Учебная database/requests.db явно включена, поскольку требуется лабораторной №5.

| Команда | Назначение |
|---|---|
| git status | Проверить состояние рабочего каталога |
| git add <файл> | Подготовить выбранные изменения |
| git commit -m "..." | Сохранить отдельный этап |
| git switch -c develop | Создать и открыть рабочую ветку |
| git merge develop | Слить develop в текущую main |
| git log --graph --all | Проверить историю и слияния |
| git push -u origin main | Отправить основную ветку после настройки URL |

Команды reset --hard и push --force не применялись, поскольку могут удалить изменения или перезаписать историю.

# 2. Коммиты и ветки

В истории более пяти осмысленных коммитов. Отдельно зафиксированы .gitignore, структура, README, ТЗ, Use Case, сравнение промптов, workflow, архитектура, интерфейс и база данных.

**Git — история коммитов и слияние**

```
PS C:\...\RequestSystem> git log --graph --all --oneline --decorate -n 18
* 4e37ee7 (HEAD -> main) feat: add normalized SQLite database and verified queries
* b05af1e docs: add seven-screen UI prototype
* 7b90303 docs: add layered architecture and component diagrams
*   b928ac5 merge: integrate develop and resolve workflow conflict
|\
| * 61728ec (develop) docs: clarify develop workflow
* | 450338c docs: define stable main branch
|/
* 4f459e4 docs: describe Git workflow
* 7333edd docs: compare AI prompts for requirements
* e2df571 docs: add use case diagram
* 771319f docs: add technical specification
* e03f589 docs: update README
* 1bdcd59 chore: add project structure
* 0048f95 chore: add gitignore
```

Фактическая история коммитов

Ветка develop создана командой git switch -c develop. В ней изменено описание workflow и выполнен коммит 61728ec. На main независимо изменён тот же фрагмент в коммите 450338c, после чего выполнено слияние.

**Git — ветки main и develop**

```
PS C:\...\RequestSystem> git branch -avv && git status
  develop 61728ec docs: clarify develop workflow
* main    4e37ee7 feat: add normalized SQLite database and verified queries

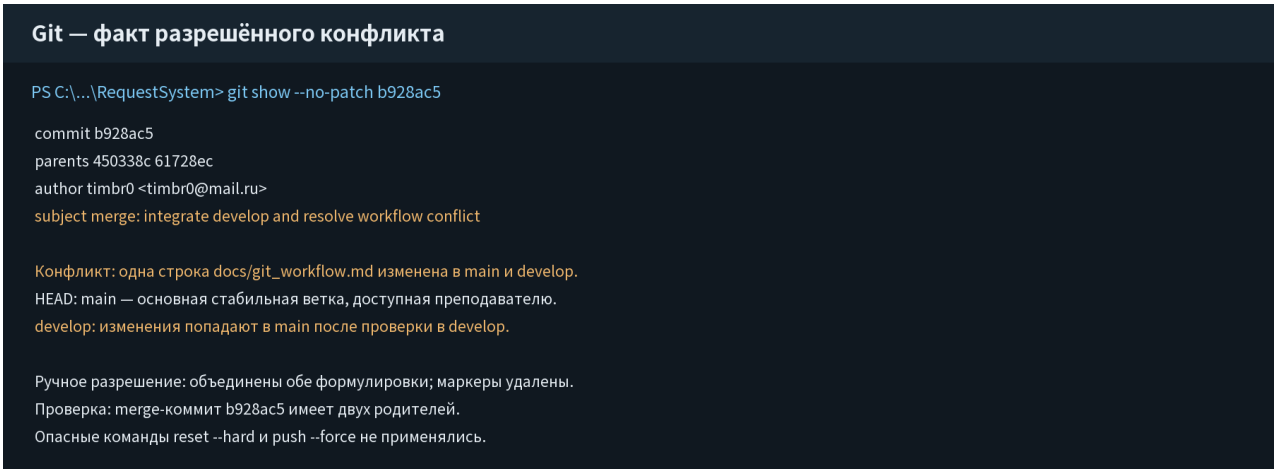
## main
?? docs/screenshots/git_branches.png
?? docs/screenshots/git_conflict.png
?? docs/screenshots/git_history.png
?? scripts/generate_git_evidence.py
user.name=timbr0
user.email=timbr0@mail.ru
```

Ветки main и develop

# 3. Конфликт и ручное разрешение

При git merge develop Git сообщил: CONFLICT (content): Merge conflict in docs/git_workflow.md. Причина — одна строка получила разные дополнения в двух ветках.

В main было указано, что это основная стабильная ветка, доступная преподавателю. В develop добавлено правило, что изменения попадают в main после проверки. Я открыл маркеры <<<<<<< HEAD, =======, >>>>>>> develop и вручную объединил обе формулировки. Автоматический выбор одной стороны удалил бы полезное правило.




Доказательство merge-коммита и разрешения

После редактирования файл добавлен через git add, создан merge-коммит b928ac5. Проверка показывает двух родителей: 450338c и 61728ec. Рабочее дерево после фиксации материалов очищено.

# 4. AI-журнал и итог

**Модель:** Kimi (Kimi Work); версия не отображается. **Задача:** получить безопасный Git-workflow, команды, сообщения коммитов, .gitignore и способ разбора конфликта. **Промпт:** полный текст сохранён в docs/ai_log.md, раздел лабораторной №2. **Почему такой запрос:** требовались не только команды, но и риски merge, reset, push --force, а также локальная настройка автора. **Результат ИИ:** рекомендован workflow main/develop, префиксы сообщений и последовательность команд.

**Проверка:** все команды выполнены локально. Фактические ветки, конфликт, merge и авторство подтверждены снимками репозитория.

**Собственная доработка:** вместо git checkout -b использовано современное git switch -c; в .gitignore добавлено исключение для сдаваемой учебной базы; конфликт разрешён объединением смысла двух версий.

**Удалённый репозиторий:** адрес GitHub будет добавлен после создания репозитория через открытое окно браузера. Локальная часть и подготовка веток выполнены полностью.

**Вывод:** я научился сохранять работу отдельными этапами, использовать ветки и разрешать конфликт без потери полезных изменений. Команды ИИ оказались полезны только после проверки на фактической истории.